

Labo 03 - Système hospitalier

Général

- Auteurs : Fabien Léger & Aymeric Bonny
- Cours : PCO, HEIG-VD

Problème

Le programme donné est une simulation de système hospitalier. Nous avons les entités suivantes:

- Ambulance : emmener les patients aux hopitaux.
- Hospital : garder les patients en attente de transfert vers les cliniques ou pour une rehabilitation.
- Clinic : soigner les patients reçus des hopitaux.
- Supplier : vendre des outils nécessaires pour soigner les patients aux cliniques.
- Insurance : payer diverses entités.

On a ainsi chaque thread qui représente une entité. On peut avoir plusieurs ambulances, hopitaux, cliniques et fournisseurs qui interagissent entre-eux en même temps. Cela pose des problèmes de concurrence. Il n'y a qu'une seule assurance dans notre cas précis.

Implémentation

endService

La seule chose à faire est de stopper les threads. On doit donc loop sur les threads et les request de stop. Ensuite, à l'intérieur des threads dans leur fonction run(), il faut checker si on nous demande de s'arrêter et si oui break pour sortir du while true.

Ambulance

Dans ambulance.cpp, il y a deux ressources à protéger. La première est **money**, dont le fonctionnement est assez conventionnel. Comme nous ne savons pas quand l'assurance nous remboursera, nous ajoutons un mutex aux attributs de la classe de manière à avoir un mutex par instance d'ambulance (money est unique à chaque ambulance) et nous protégeons tous les accès à cette ressource.

La deuxième ressource à protéger est un peu plus intéressant puisqu'elle est partagée entre toutes les ambulances; il s'agit du **stocks** de patients malades. Pour protéger cette ressource, nous créons donc un mutex static qui est donc unique et partagé parmi toutes les ambulances.

Hospital

Dans `hospital.cpp`, il y a trois éléments à protéger à l'aide de mutex. Etant donné que nous ne savons pas quand l'assurance pourrait nous rembourser, il nous faut protéger toutes les manipulations concernant **money**, y compris sa lecture, notamment lorsqu'une condition en dépend.

Pour les mêmes raisons, nous devons aussi protéger les **sickPatients** contenus dans `stocks`, et les **rehabPatients** également contenu dans `stocks`. En effet, nous savons quand l'hôpital se charge de transmettre les patients malades aux cliniques, mais nous ignorons complètement quand les ambulances viendront nous les amener, et nous ne savons pas non plus quand les cliniques nous les ramèneront. Ce sont donc des mutex rajoutés en attribut à la classe `Hospital`.

Pour les choix d'implémentations ne concernant pas les mutex, les deux fonctions dignes d'intérêt sont `updateRehab` et `transfer`. Dans `transfer`, il nous faut identifier le type de patient qui nous est transféré, de manière à utiliser le bon mutex. Pour cette raison, nous utilisons un switch qui exécute le code correspondant à la nature du transfert. Quant à la fonction `updateRehab` c'est probablement la plus complexe de la classe. Pour déterminer quand un patient atteint la fin de sa période de convalescence, nous avons créé un tableau à 4 cases (convalescence de 5 jours exclu) qui sera rempli avec le nombre de patients réhabilités arrivés chaque jour à l'hôpital. Nous indexons ensuite sur la taille du tableau pour connaître le nombre exact de patients à libérer, et remplacer cette valeur par le nombre de nouveaux `rehabPatients` arrivés aujourd'hui.

Clinic

Il faut protéger **money** avec un mutex. On peut recevoir de l'argent d'assurance en même temps qu'on veut payer nos factures ou alors qu'on veut traiter nos patients. La section critique dans `treatOne()` est longue, car on doit prendre une décision sur l'argent avant de faire tous nos changements puis `unlock()` le mutex.

Il est également bon d'avoir un mutex pour **stocks** sous la forme de `patientsMutex`. Il est seulement nécessaire de protéger quand on veut nous donner des patients et quand on veut s'occuper de ses patients ou les renvoyer aux hopitaux. Il n'y a pas besoin pour les commandes de matériel car ceci se fait uniquement via l'instance.

Nous n'avons pas compris à quoi sert `sickQueue` vu qu'on a déjà les `SickPatient` et `RehabPatient` dans `stocks`.

Supplier

Il est nécessaire d'avoir une protection sur **money**. On peut lorsqu'on produit une ressource avoir un hopital qui nous paie.

Il faut également avoir un mutex pour **stocks**. On peut lorsqu'on produit une ressource avoir un hopital qui nous en achète.

Insurance

Il faut protéger **unpaidBills**, car on peut recevoir des invoice() de la part des autres sellers mais également on veut que notre instance utiliser payBills(). Grâce à billMutex, il est possible de protéger ces sections critiques. Dans payBills, la section critique est assez grande, parce qu'on avance dans unpaidBills à chaque tour de boucle. Ainsi, il paraît bon de lock au cas où il y aurait des réactions non-définies. Typiquement, le vecteur se faisant réallouer, car dépassant sa capacité ce qui casserait notre itérateur.

Il n'y a pas besoin de protéger money, car celle-ci est toujours appelée par l'instance d'Insurance. En effet, soit on fait receiveContributions, soit on fait payBills, mais jamais les deux.

Tests

Les tests ayant déjà été mis en place au préalable, nous n'en avons pas fait plus. Nous passons tous les tests donc, cela est déjà un bon début. Bien évidemment, ces tests ne vérifient pas tout.

Fabien Léger & Aymeric Bonny